

## *Bài 10*

# **Ngôn ngữ C Hàm**

## **Nội dung**

---

- Hàm
- Gọi hàm bằng giá trị
- Gọi hàm bằng tham chiếu
- Đối số là mảng
- Cơ chế kết thúc của hàm
- Nguyên mẫu hàm

## Hàm - Định nghĩa

---

- Trong ngôn ngữ C, hàm là một khối xây dựng cơ bản của một chương trình C cung cấp tính mô-đun.
- **Định nghĩa của một hàm** là khối lệnh có dạng  
*<kiểu trả về> <tên hàm>(<danh sách tham số>)*  
{  
    *<các lệnh phần thân hàm>*  
}
- *<kiểu trả về>* là kiểu dữ liệu mà hàm trả về. Một hàm có thể trả về bất kỳ kiểu dữ liệu nào, ngoại trừ kiểu mảng.
- *<danh sách tham số>* là danh sách các biến được khai báo để nhận đối số khi hàm được gọi thi hành.

## Hàm - Phạm vi

---

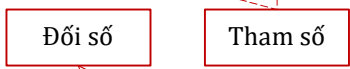
- Định nghĩa của hàm xác định phạm vi khối.
  - Biến được khai báo trong hàm là biến cục bộ. Nó được tạo ra khi hàm thi hành và bị hủy khi hàm kết thúc.
  - Các tham số của một hàm cũng được dùng như biến cục bộ của hàm.
- Các lệnh phần thân của một hàm là riêng tư, chúng chỉ được thi hành thông qua lời gọi hàm đó.
- Các hàm có phạm vi tập tin.
  - Hàm có thể được gọi ở bất kỳ vị trí nào sau vị trí nó được khai báo nguyên mẫu hoặc định nghĩa.

## Hàm - Đối số của hàm

- Khi một hàm được gọi thi hành, nếu nó cần truyền vào các mục dữ liệu thì
  - Các mục dữ liệu đó được gọi là **đối số của hàm**.
  - Định nghĩa hàm phải khai báo các **tham số** để nhận **đối số**.

```
// Hàm tính diện tích hình chữ nhật
double computeRectangleArea(double length, double width) {
    double area = length * width;
    return area;
}

void main() {
    double area = computeRectangleArea(3.0, 2.0);
    printf("Dien tich hcn = %lf\n", area);
}
```



## Hàm - Lời gọi hàm

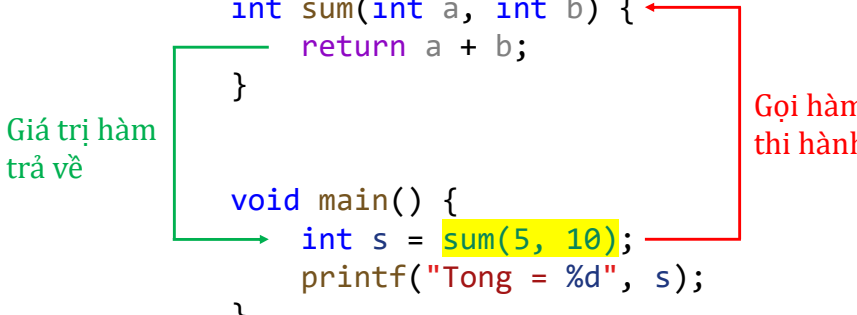
- **Lời gọi hàm** (function call) là biểu thức để chỉ thị cho hàm thi hành.

*<tên hàm>([danh sách đối số])*

- Cơ chế thi hành của hàm

```
int sum(int a, int b) {
    return a + b;
}

void main() {
    int s = sum(5, 10);
    printf("Tong = %d", s);
}
```



## Gọi hàm bằng giá trị

- Truyền đối số cho hàm bằng giá trị:
  - Giá trị đối số được sao chép vào tham số tương ứng.
  - Bên trong hàm, những thay đổi được thực hiện với tham số không ảnh hưởng đến đối số.
  - Tham số phải khai báo là biến thông thường.

```
int sum(int a, int b) {  
    return a + b;  
}  
  
void main() {  
    int a = 5, b = 10;  
    printf("Tong = %d", sum(a, b));  
}
```

Tham số là biến thông thường

Giá trị của đối số

## Gọi hàm bằng tham chiếu

- Truyền đối số cho hàm thông qua địa chỉ:
  - Địa chỉ đối số được sao chép vào tham số tương ứng.
  - Bên trong hàm, những thay đổi được thực hiện với tham số sẽ ảnh hưởng đến đối số.
  - Tham số phải khai báo là biến con trỏ.

```
void add(int *n, int i) {  
    *n = *n + i;  
}  
  
void main() {  
    int a = 5;  
    add(&a, 5);  
    printf("a = %d", a);  
}
```

Tham số là biến con trỏ

Địa chỉ của đối số

## Đối số là mảng (1)

- Khi dùng mảng là đối số thì
  - Địa chỉ của mảng sẽ được truyền cho hàm thay vì giá trị.
  - Mã bên trong hàm có khả năng thay đổi giá trị của mảng.
  - Mảng được truyền bằng địa chỉ sẽ bị mất thông tin về kích thước. Do đó phải truyền thêm giá trị kích thước.
- Các tham số để nhận đối số là mảng gồm
  - Tham số nhận địa chỉ mảng phải khai báo là biến con trỏ có kiểu của phần tử thứ 0 hoặc phần tử nguyên tử.
  - Tham số nhận kích thước mảng phải khai báo là biến thông thường kiểu int.

```
// Hàm nhận đối số là mảng một chiều  
void input(int arr[], int n);  
void input(int *pArr, int n);
```

Tham số nhận  
địa chỉ mảng

Tham số nhận  
kích thước

## Đối số là mảng (2)

- Bảng các tham số cần để nhận đối số là mảng

| Dạng đối số                | Tham số<br>nhận địa chỉ                                                                        | Tham số<br>nhận kích thước                                |
|----------------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Mảng một chiều<br>kiểu int | int arr[]<br>hoặc<br>int *arr                                                                  | int n<br>(số phần tử)                                     |
| Mảng hai chiều<br>kiểu int | int arr[][<hằng nguyên>]<br>(hằng nguyên cho cột)<br>hoặc<br>int *arr                          | int m, int n<br>(kích thước hàng,<br>cột)                 |
| Mảng ba chiều kiểu<br>int  | int arr[][<hằng nguyên>]<br>[<hằng nguyên>]<br>(hằng nguyên cho hàng, cột)<br>hoặc<br>int *arr | int l, int m,<br>int n<br>(kích thước tầng,<br>hàng, cột) |

## Đối số là mảng một chiều

```
#define NUM_OF_ELEM 5

// Ham nhap mang 1 chieu cac so nguyen
void inputArr1D(int arr[], int n) {
    printf("Nhap mang 1 chieu cac so nguyen:\n");
    for (int i = 0; i < n; i++) {
        printf("%4cPhan tu thu %d = ", 32, i);
        scanf("%d", &arr[i]);
    }
}

void main() {
    int a[NUM_OF_ELEM];
    inputArr1D(a, NUM_OF_ELEM);
}
```

Tham số nhận địa chỉ mảng 1 chiều:  
- Con trỏ trỏ tới phần tử thứ 0  
- **Tương ứng kiểu int \***

Đối số là địa chỉ của phần tử thứ 0.

## Đối số là mảng hai chiều

```
#define NUM_OF_ROW 2
#define NUM_OF_COL 4

// Ham nhap mang 2 chieu cac so nguyen
void inputArr2D(int arr[][NUM_OF_COL], int m, int n) {
    printf("Nhap mang 2 chieu cac so nguyen:\n");
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++) {
            printf("%4cPhan tu thu [%d][%d] = ", 32, i, j);
            scanf("%d", &arr[i][j]);
        }
}

void main() {
    int b[NUM_OF_ROW][NUM_OF_COL];
    inputArr2D(b, NUM_OF_ROW, NUM_OF_COL);
}
```

Tham số nhận địa chỉ mảng 2 chiều:  
- Con trỏ trỏ tới phần tử thứ 0  
- **Tương đương kiểu int (\*)[4]**

Đối số là địa chỉ của phần tử thứ 0.

## Cơ chế kết thúc của hàm

- Một hàm sẽ kết thúc thi hành và chuyển điều khiển quay về vị trí hàm được gọi khi câu lệnh cuối cùng được thực hiện hoặc gặp lệnh return đầu tiên.
- **Lệnh return** để kết thúc việc thi hành của hàm.  
*return <biểu thức>;*
- Nếu hàm có kiểu trả về là void thì lệnh return có thể bỏ qua hoặc dùng với *<biểu thức>* rỗng.
- Nếu hàm có kiểu trả về khác void thì phải có lệnh return và giá trị trả về của hàm là kết quả của *<biểu thức>*.
- Trong một hàm có thể có nhiều lệnh return, nhưng phải đảm bảo logic chỉ có một lệnh return xảy ra.

## Nguyên mẫu hàm

- Nguyên mẫu hàm là tiêu đề của hàm gồm tên hàm, kiểu trả về và các tham số.  
*<kiểu trả về> <tên hàm>(<danh sách tham số>)*
- Khai báo nguyên mẫu hàm là lệnh thông báo cho trình biên dịch biết có sự tồn tại của hàm trong chương trình.
- Sử dụng khai báo nguyên mẫu hàm khi lời gọi hàm xuất hiện trước định nghĩa của hàm.
- Nguyên mẫu hàm không thể thay thế cho định nghĩa của hàm.

## Nguyên mẫu hàm - Ví dụ

```
// khai báo nguyên mẫu của hàm sum
int sum(int a, int b);

void main() {
    int a = 5, b = 10;
    printf("Tong = %d", sum(a, b));
}

// định nghĩa của hàm sum
int sum(int a, int b) {
    return a + b;
}
```

Lời gọi hàm xuất hiện trước định nghĩa hàm